

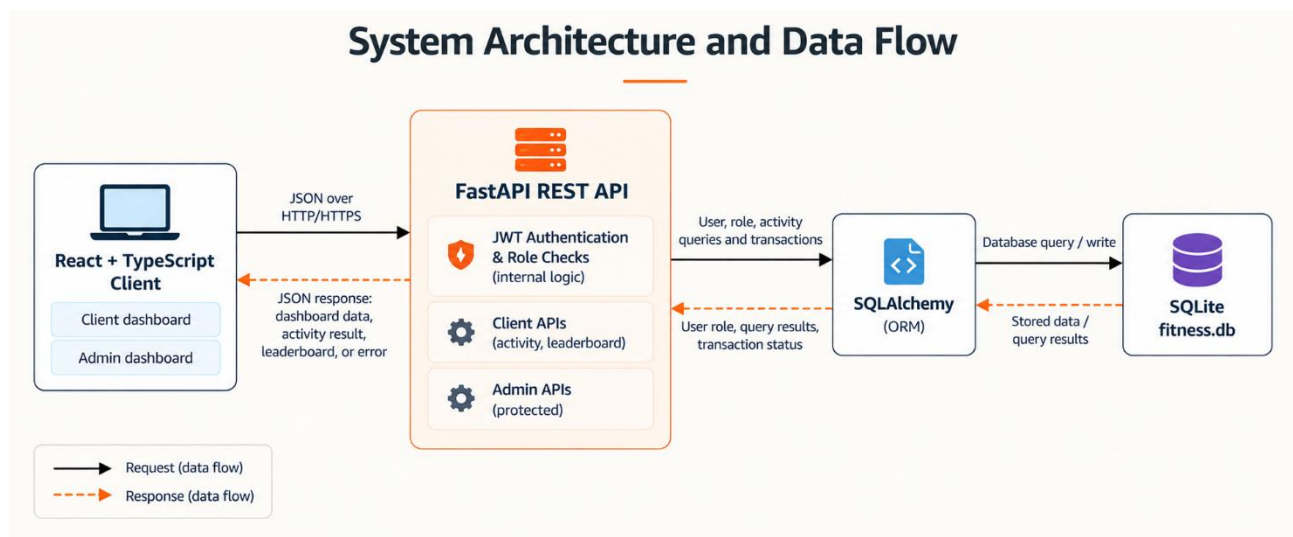
# FitChallenge Software Design Document

## 1. Scope

FitChallenge is a fitness activity tracker. Clients register, record an activity for a date, receive normalized points, and use a personal dashboard and global leaderboard. A separately provisioned administrator can view client-only aggregate data, client accounts, rankings, and activity history.

The system uses a React/TypeScript frontend, FastAPI backend, and persistent SQLite database.

## 2. System Architecture and Data Flow



The frontend uses Axios to attach the JWT bearer token to authenticated requests. The API is the source of truth for registration, roles, activity validation, scoring, and authorization. The browser may redirect users to an appropriate route for usability, but administrator access is always checked by the backend before protected data is returned.

### Registration flow

A new user submits their username, email, password, first name, and last name through the public registration form. The React frontend sends this data to 'POST /api/auth/register' as JSON.

FastAPI validates the request fields and email format. It checks whether the username, email, or normalized first-name and last-name combination already exists. To prevent concurrent duplicate registrations, the backend creates a unique normalized-name reservation in the 'registered\_names' table.

The password is hashed with bcrypt before any user record is stored. The backend creates the account with the server-assigned role 'client'; public registration cannot create an administrator account. SQLite allocates the internal numeric primary key, after which the backend creates a unique public identifier in the format 'USR###', such as 'USR005'.

The transaction is committed only when all checks succeed. The API returns the new user profile, including both the internal 'id' and public 'userId'. If duplicate or invalid data is submitted, the API returns an appropriate validation error. Database constraints protect the system if duplicate requests

arrive at the same time.

## Activity ingestion flow

An authenticated client selects an activity type, enters its matching metric value, and chooses an activity date in the Record Activity page. The frontend prevents selection of future dates and sends the activity details to `POST /api/activities` with the user's JWT bearer token.

FastAPI validates the JWT and retrieves the account role. Only accounts with `role = client` can record activities; administrator accounts receive `403 Forbidden`. The backend validates that the activity type is supported, the metric is greater than zero, the value follows the correct numeric format, and the value does not exceed the activity-specific maximum. Swimming, gym, and daily-step values must be whole numbers.

The backend checks for an existing entry with the same user, activity type, and activity date/time. It inserts an `activity\_submissions` reservation and the workout record within the same transaction. The database-level unique constraint prevents duplicate concurrent submissions.

After saving the workout, FastAPI calculates points using the relevant normalization and flooring rule. The response includes the activity identifier, activity type, entered value, calculated points, and activity date. The frontend displays the awarded points and refreshes the relevant dashboard and leaderboard data.

## 3. Database Schema and Data Model

FitChallenge uses SQLite (fitness.db) with SQLAlchemy ORM models. The system stores users and activity records. The leaderboard is a derived view calculated from client activity records; it is not stored as a separate database table.

`users`

| Column          | Type     | Constraints / purpose                                                  |
|-----------------|----------|------------------------------------------------------------------------|
| id              | INTEGER  | Primary key; internal numeric identifier                               |
| user_id         | TEXT     | Unique public identifier, generated as USR001, USR002, etc.            |
| username        | TEXT     | Unique                                                                 |
| email           | TEXT     | Unique                                                                 |
| hashed_password | TEXT     | Stores bcrypt password hash only; plaintext passwords are never stored |
| first_name      | TEXT     | User first name                                                        |
| last_name       | TEXT     | User last name                                                         |
| role            | TEXT     | Server-assigned role: client or admin                                  |
| created_at      | DATETIME | Account creation timestamp                                             |

## `workouts`

| Column        | Type     | Constraints / purpose                                          |
|---------------|----------|----------------------------------------------------------------|
| id            | INTEGER  | Primary key                                                    |
| user_id       | INTEGER  | Foreign key referencing users.id                               |
| activity_type | TEXT     | Running, Walking, Cycling, Swimming, Gym, or Daily Steps       |
| distance      | FLOAT    | Distance in kilometres; used for running, walking, and cycling |
| duration      | FLOAT    | Duration in minutes; used for swimming and gym                 |
| steps         | INTEGER  | Step count; used for Daily Steps                               |
| recorded_at   | DATETIME | Activity date selected by the user                             |
| created_at    | DATETIME | Database record creation time                                  |

## Duplicate-user enforcement

1. The API first checks `username`, `email`, and the standardized first-name and last-name pair so it can return a clear validation error.
2. The `users` table independently enforces uniqueness for `username`, `email`, public `user\_id`, and the pair (`first\_name`, `last\_name`).
3. During registration, the backend trims extra whitespace and applies a title case before storing `first\_name` and `last\_name`. If duplicate registrations pass the initial checks, the database constraint rejects one transaction; the backend rolls it back and returns `409 Conflict`.

## 4. API Specifications

All JSON requests use `Content-Type: application/json`. Protected endpoints require `Authorization: Bearer <JWT>`.

### Authentication and account APIs

| Route                      | Request                                                          | Success response                                                                                   | Validation and errors                                                                           |
|----------------------------|------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| POST<br>/api/auth/register | { username,<br>email,<br>password,<br>first_name,<br>last_name } | User object including numeric id, public userId such as USR005, profile fields, and role: "client" | Rejects duplicate username/email/full name (400 or 409). Role is not accepted from the request. |

|                             |                           |                                                                      |                                                                 |
|-----------------------------|---------------------------|----------------------------------------------------------------------|-----------------------------------------------------------------|
| POST<br>/api/auth/login     | { username,<br>password } | { access_token,<br>token_type,<br>user_id,<br>username }             | Invalid credentials<br>return 401.                              |
| GET<br>/api/users/me        | JWT                       | Current user with id,<br>userId, profile and<br>backend-sourced role | Missing/invalid JWT<br>returns 401.                             |
| GET<br>/api/users/{user_id} | Path ID                   | Client user profile                                                  | Admin accounts return<br>404 from this client-<br>facing route. |

After login, the frontend obtains /api/users/me; the returned role selects the client or administrator dashboard. This is not a security boundary: the API checks the same role from the database for every protected admin request.

## Activity APIs

| Route                      | Request or response                           | Rules                                                                                                              |
|----------------------------|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| POST /api/activities       | { activity_type,<br>value,<br>activity_date } | Client role required. Returns { id, activity_type, value, points, recorded_at }. Duplicate submission returns 409. |
| GET /api/activities        | Client's own activity array                   | Client role required. Admin accounts receive 403.                                                                  |
| GET /api/leaderboard       | Ranked client totals                          | Excludes admin accounts. No JWT is required for the global read model.                                             |
| GET /api/leaderboard/trend | Current client's date/rank series             | Client role required.                                                                                              |

Allowed activity types are running, walking, cycling, swimming, gym, and daily\_steps. Values must be positive. Swimming, gym, and daily steps must be whole numbers. Reasonable maximums are enforced: running 200 km, walking 100 km, cycling 500 km, swimming/gym 1440 minutes, and daily steps 100,000.

## Administrator APIs

| Route                   | Response                                                            | Authorization                                    |
|-------------------------|---------------------------------------------------------------------|--------------------------------------------------|
| GET /api/admin/overview | Client totals, seven-day user-growth series, recent client activity | role = admin required; otherwise 403.            |
| GET /api/admin/users    | Client accounts with activity count and points                      | Admin only; admins are excluded from the result. |

| Route                        | Response                    | Authorization                                    |
|------------------------------|-----------------------------|--------------------------------------------------|
| GET<br>/api/admin/activities | All client activity entries | Admin only; admins are excluded from the result. |

The admin dashboard's **View All** link routes to /admin/activities, which uses the final endpoint. The backend returns 403 Forbidden for a valid client token at any /api/admin/\* endpoint.

## 5. Scoring and Normalization Logic

Points are calculated when an activity is read or returned. The same function is used by activity responses, dashboards, and leaderboard aggregation.

| Activity    | Input metric | Rule                                 | Points                                   |
|-------------|--------------|--------------------------------------|------------------------------------------|
| Running     | km           | Floor fractional result              | $\text{floor}(\text{km} \times 100)$     |
| Walking     | km           | Floor fractional result              | $\text{floor}(\text{km} \times 50)$      |
| Cycling     | km           | Floor fractional result              | $\text{floor}(\text{km} \times 25)$      |
| Swimming    | minutes      | Count completed whole minutes only   | $\text{floor}(\text{minutes}) \times 15$ |
| Gym         | minutes      | Count completed whole minutes only   | $\text{floor}(\text{minutes}) \times 5$  |
| Daily Steps | steps        | Count completed 100-step blocks only | $\text{floor}(\text{steps} / 100)$       |

Examples:

- 1.55 km walking  $\rightarrow \text{floor}(77.5) = \mathbf{77 \text{ points}}$ .
- 1 minute 55 seconds gym input is validated as whole minutes; 1 minute  $\rightarrow \mathbf{5 \text{ points}}$ .
- 399 daily steps  $\rightarrow \text{floor}(399 / 100) = \mathbf{3 \text{ points}}$ .

## Ranking algorithm

For every client, valid activity points are cumulatively summed. Totals are sorted by descending points, then by internal user ID for deterministic output. Competition ranking is applied: equal totals receive the same rank and the next rank skips positions (1, 2, 2, 4). The client leaderboard refreshes on its existing 15-second frontend interval; each refresh fetches a newly derived server total. The rank trend recomputes historical cumulative rank by date from saved client activities.

## 6. Frontend Architecture and Visualizations

### Component and route breakdown

| Area                  | Main frontend pieces                                                               | Behavior                                                                                       |
|-----------------------|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Public pages          | HomePage, LoginPage, RegisterPage                                                  | Registration/login; role is never sent by the client.                                          |
| Client workspace      | ReferenceDashboardPage, ValidatedRecordActivityPage, ReferenceLeaderboardPage      | Dashboard, activity entry, and leaderboard routes are wrapped in ClientRoute.                  |
| Admin workspace       | AdminDashboardPage, AdminUsersPage, AdminLeaderboardPage, AdminActivityHistoryPage | Routes are wrapped in AdminRoute; the sidebar changes by role.                                 |
| Shared infrastructure | authStore, apiClient, Navbar, Sidebar, route guards                                | Stores user/role and JWT locally, attaches the bearer token, and handles presentation routing. |

The personal dashboard derives today's points, streak, calendar marks, selected date history, personal-best values, activity volume, and sport preference from the client's `/api/activities` data. It displays an empty state rather than a chart when there is no activity.

The activity volume chart is an SVG line chart whose selected y-axis is steps, distance, or duration over seven days. The sport-preference chart is an SVG pie chart; hover reveals the selected sport and percentage. The leaderboard and ranking trend use SVG-based visualizations and API-derived ranked totals.

## 7. Trade-offs and Edge Cases

| Decision / edge case                | Handling and rationale                                                                                                                                           |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SQLite instead of a server database | Simple persistent local deployment. SQLAlchemy isolates the persistence code, allowing a later move to PostgreSQL if multi-process production scale is required. |
| Duplicate registration requests     | Application checks give user-friendly messages; unique database constraints and transactional name reservations protect concurrent requests.                     |
| Duplicate activity requests         | Application pre-check catches ordinary repeats; the unique workout tuple protects simultaneous identical requests.                                               |
| Invalid or missing metric           | Frontend shows required/format errors; FastAPI validates type, positivity, whole-number-only                                                                     |

| Decision / edge case     | Handling and rationale                                                                                                                 |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
|                          | metrics, and maxima.                                                                                                                   |
| Zero or negative metric  | A clear validation dialog is shown in the form and API rejects the request.                                                            |
| Future activity date     | The date input has a maximum of today, preventing upcoming dates in the UI.                                                            |
| Decimal scoring          | Distance points are floored at the final points calculation; duration and step rules floor completed units/blocks.                     |
| No activity              | Client dashboard shows zero-valued summary data and no-chart empty states.                                                             |
| Tied totals              | Competition ranking preserves ties as 1, 2, 2, 4.                                                                                      |
| Admin account visibility | Admin records are filtered from client lists, leaderboard calculation, client activity APIs, admin user list, and admin activity list. |
| Frontend route tampering | AdminRoute improves navigation, but FastAPI role checks are decisive. A client token receives 403 from admin APIs.                     |
| Administrator password   | Provisioning hashes the password with bcrypt immediately. No plaintext password is written to SQLite or returned by an API.            |